A Hybrid Zero Trust Architecture for Non-Interactive Authentication: Integrating Hardware Trust Anchors with Software-Defined Secret Management in Infrastructure as Code

Juliano S. Langaro, Altair O. Santin, Eduardo K. Viegas, Fellipe M. Veiga, and Juarez Oliveira
Graduate Program in Computer Science (PPGIa), PUCPR, Curitiba, Brazil

***Abstract*—**Non-Interactive Authentication (nIA)—the machine-to-machine authentication that underpins Infrastructure as Code (IaC) pipelines and unattended IoT fleets—is chronically undermined by the *Secret Zero* problem: a long-lived credential must be planted in software to bootstrap every other secret, and that credential is extractable under OS compromise (MITRE ATT&CK T1003). This paper presents a hybrid Zero Trust architecture that anchors secret management in hardware trust anchors (TPM 2.0, with an upgrade path to Intel SGX and TDX), delegates HashiCorp Vault auto-unseal to a physical TPM, and confines every credential inside an identity-based network perimeter (ZTNA) so intercepted secrets cannot be replayed outside their network context. We describe a publicly released reference implementation whose IoT agent seals a per-device one-time-password seed inside the device TPM while registering the same seed in the server's Vault, so the plaintext seed never touches disk on either side. We evaluate the design along three axes new to this revision: (i) provisioning time for the full infrastructure, (ii) secret-retrieval latency on the hot path, and (iii) resistance to an on-premises LLM red-team mapped to MITRE ATT&CK. We further contribute a proposed security-test suite that turns the ad-hoc red-team into a repeatable regression gate executed after each `terraform apply`. Measured software-path costs are sub-25 microseconds; the hardware path adds a bounded, single-digit-to-tens-of-milliseconds overhead while reducing Vault RTO from minutes to milliseconds. Results are indicative estimates backed by prototype measurements; production-scale empirical validation remains future work.

***Index Terms*—**Zero Trust, Trusted Platform Module, HashiCorp Vault, non-interactive authentication, Infrastructure as Code, MITRE ATT&CK, IoT security, confidential computing, TOTP, LLM red-teaming.

I. INTRODUCTION

MODERN infrastructure is provisioned by code and operated by machines. IaC engines apply changes without a human at the keyboard, and IoT fleets authenticate at boot with no operator present. This non-interactive setting removes the human factor that interactive MFA relies on and forces a bootstrap credential—the Secret Zero—to live in software. Once an adversary attains OS privileges, that credential and any secret it unlocks become extractable (T1003 OS Credential Dumping; T1552 Unsecured Credentials).

Software-only secret managers such as HashiCorp Vault mitigate sprawl but do not by themselves solve Secret Zero: Vault must itself be unsealed, and the unseal material is the new Secret Zero. Prior work by Oliveira *et al.* [12] showed that a non-interactive OTP method materially raises the cost of Vault credential abuse; this paper generalizes and hardens that line of work by moving the trust anchor into silicon and enclosing the entire flow in an identity-based network perimeter.

Contributions. This revision consolidates and extends our earlier results: (1) a three-layer hybrid ZT design combining a hardware root of trust (TPM 2.0, with SGX/TDX tiers), software-defined secret management (Vault auto-unseal), and identity-based network access (ZTNA via an on-premises Twingate connector); (2) a hardware-anchored OTP scheme in which the shared seed is sealed inside the TPM and is non-exportable; (3) a protocol-agnostic two-channel MFA model for unattended IoT over REST/HTTPS and MQTT/TLS; (4) an IaC-driven tiered protection model (TPM/SGX/TDX) selected via Terraform sensitivity labels; (5) a reproducible on-premises LLM adversary simulation mapped to MITRE ATT&CK; and—new—(6) a provisioning-time and secret-retrieval-latency characterization of the running prototype, plus (7) a proposed security-test suite that converts the red-team into a post-deployment regression gate.

II. THREAT MODEL

We assume a root-level software adversary: an attacker with privileged code execution on a host who cannot physically decap or glitch the TPM silicon. The adversary's goals and MITRE ATT&CK techniques are: credential extraction (T1003, T1552); network interception (T1040 Network Sniffing); replay/token reuse (T1550, T1078); privilege escalation (T1068, TA0004); and lateral movement (T1021, TA0008). Physical TPM attacks (cold-boot, bus probing, decapping) are out of scope; we rely on certified TPM 2.0 tamper-resistance [13] and note the residual risk in Section VII.

TABLE I
Positioning Against Closely Related Work

Approach	HW anchor	Vault/secret	ZT net
Perimeter VPN	no	partial	no
Vault-only (software)	no	yes	no
Oliveira *et al.* [12]	partial (OTP)	yes (OTP-hard.)	no
LLM pen-testing [1,2,8]	n/a	n/a	n/a
This work	**yes TPM/SGX/TDX**	**yes (auto-unseal)**	**yes**

III. ARCHITECTURE

The architecture rests on the premise that software-only security is insufficient for nIA. It is organized in three enforcement layers, each corresponding to a concrete module in the public repository.

A. Layer 1 — Hardware Root of Trust (TPM/SGX/TDX)

Keys are generated inside the TPM with `fixedtpm` and `fixedparent` so they never leave the silicon, even under full OS compromise. Sealing binds material to PCR 0 and 7, so tampered firmware or a disabled Secure Boot state prevents unsealing. Higher tiers use Intel SGX (Gramine) and Intel TDX for CPU-encrypted memory, assigned automatically by IaC.

B. Layer 2 — Software-Defined Secret Management (Vault)

Vault is initialized (`sys/init`, five Shamir shares, threshold three [11]) on first boot; unseal shares and root token are sealed under a

persistent TPM Storage Root Key. On every boot an initializer unseals the shares from the TPM and applies them via the Vault REST API (`sys/unseal`) with exponential backoff. No plaintext secret and no `vault` binary are needed in the container; no cloud KMS is involved. This eliminates Secret Zero for the secret manager and collapses RTO from a manual multi-minute unseal to sub-second.

C. Layer 3 — Identity-Based Network Access (ZTNA)

A Twingate connector runs on-premises as a Docker container making only *outbound* connections to the Twingate relay, so no inbound port is opened on production. Vault and the IoT server are published as Twingate Resources reachable only by identities satisfying an access policy (MFA, optional device posture). Even a leaked OTP cannot be used outside the ZTNA network context, neutralizing lateral movement (T1021).

D. Deployment Topology

Two virtual servers: **PPGIA96** (production) hosts Vault (`:8200`), the IoT server (REST `:5000` / MQTT `:8883`), and the Twingate connector. **PPGIA95** (testing) hosts the security-testing module and validation client, reaching PPGIA96 exclusively through Twingate. Production exposes no inbound ports, so the only path in is an authorized, policy-checked ZTNA session.

IV. IMPLEMENTATION

The prototype uses Docker Compose for orchestration, `tpm2-tools`/`tpm2-pytss` for TPM operations, and Python agents, on Ubuntu 22.04 LTS with physical (Infineon SLB9670/9665) and virtual (`swtpm`) TPMs.

A. Hardware-Anchored One-Time Passwords

The hardware path computes a counter-based OTP whose HMAC is evaluated inside the TPM (RFC 4226): HOTP(K,C) = Truncate(HMAC-SHA1$_{TPM}$(K,C)), where K is a non-exportable keyed-hash object in TPM NVRAM and C is a hardware monotonic counter, so rollback is physically prevented.

```
from tpm2_pytss import ESAPI
def generate_hardware_hotp(nv_index, key_handle):
    ctx = ESAPI()
    ctx.nv_increment(nv_index)  # monotonic counter
    counter = ctx.nv_read(nv_index)
    mac = ctx.hmac(key_handle, counter)
    return truncate_to_hotp(mac) # key stays in silicon
```

Listing 1. HMAC delegated to the TPM; the key never leaves the chip.

B. Reference IoT Flow: TOTP Seed Sealed in the TPM

For full reproducibility on commodity hardware, the released IoT agent implements a time-based OTP (TOTP, RFC 6238) as the functional analog of the HOTP above; the TPM-delegated HMAC of Listing 1 is retained for hardware deployments and is the intended production path. The released flow is protocol-agnostic (identical for REST and MQTT):

1) Provisioning (once per device, `init_device.sh`): a random Base32 TOTP seed is generated in RAM, *sealed in the device TPM* under the persistent SRK (`0x81010001`), and *registered in the server Vault* at `secret/data/tpm-verified/iot/devices/<id>` (field `otp_secret`). The plaintext seed never touches disk. **2) Client boot**: verify TPM (`tpm2_getrandom`) and unseal the seed into memory; a changed PCR/boot state fails the unseal. **3) Authenticate**: send `device_id` + current TOTP (REST `POST /verify` or MQTT `iot/verify`). **4) Server**: read the same seed from Vault, cache it, and validate with ±1-window tolerance.

This asymmetry is the key property: the client holds its seed only inside hardware (never on disk), while the server holds seeds only inside Vault (never in code), so neither endpoint stores a reusable plaintext credential at rest.

C. IaC-Driven Tiered Protection

Terraform sensitivity labels map workloads to tiers (Table II): `standard` → TPM 2.0; `high` → TPM+SGX (Gramine); `critical` → vTPM+TDX. Twingate resources and access groups are themselves declared in Terraform, so the perimeter is version-controlled alongside the compute tier.

TABLE II
Hybrid Server Protection Tiers (IaC-selected)

Label	HW anchor	TEE	Auto-unseal binding
standard	TPM 2.0	none	PCR sealing
high	TPM + SGX	Gramine LibOS	SGX + TPM
critical	vTPM + TDX	TDX Trust Dom.	vTPM PCR

V. PROPOSED SECURITY-TEST SUITE

A recurring weakness of nIA deployments is that security is validated once, by hand, and then drifts. We propose a repeatable suite run after every `terraform apply` that gates promotion to production. Family A is automated network/vuln testing driven by the `pentest/` module; Family B checks functional security invariants of the TPM+Vault+ZTNA flow (Table III). Because Family A LLM agents run fully on-premises (`pentestv3.py`), no architectural detail leaves the environment—a prerequisite for a federal-court network.

VI. EVALUATION

Scope disclaimer. Results combine software simulation (`swtpm`, Ollama), prototype measurements on a controlled testbed, and analytical estimation—indicative estimates, not confirmed production benchmarks. Directly measured values are marked **(m)**; analytically estimated values **(e)**.

A. Automated Adversary Simulation via Local LLM Agents

Following autonomous pen-testing frameworks [1,2,8], we drive a local LLM adversary (Llama 3.1 via Ollama) with two tools (`http_request`, `encode_payload`), a 5–10 turn limit, and n=100 independent runs per technique. Against the hybrid architecture the observed bypass rate is 0% across five techniques (5×100 = 500 runs total), 95% Wilson upper bound 3.6% per technique; the software-only baseline averages 18.4% (Table IV).

Two threat-model techniques—T1040 (sniffing) and T1550 (replay)—are addressed by TLS transport and by invariants B2/B3/B7 of Table III rather than by the LLM agent, reconciling the threat model with the simulated techniques.

B. IoT Prototype and TPM Conformance

Table V reports IoT results on Raspberry Pi and under `swtpm`; TPM conformance (SHA-256, HMAC, NV counters, PCR ops) was validated with ELTT2.

C. Provisioning Time of the Full Infrastructure (new)

We characterize the time to bring the PPGIA96 stack from bare Docker to a healthy, unsealed state: the `vault-tpm` services (`vault:1.13.3` plus initializer and TPM-validator builds), the IoT server (REST build, or Mosquitto + MQTT build), and the Twingate connector. Table VI decomposes the budget. TPM auto-unseal is the decisive win: it removes the manual multi-minute unseal from the critical path, reducing Vault RTO to sub-second (Table VIII).

D. Secret-Retrieval Latency on the Hot Path (new)

The server obtains a seed via the cache → Vault KV v2 → `.env` fallback chain of `load_device_secret`. We measured the software components directly (20,000 iterations) and estimate the cold Vault round-trip from typical `hvac` KV v2 LAN latency (Table VII). The dominant cost is one cold Vault read per device; every subsequent retrieval is a warm cache hit at sub-microsecond cost.

E. Performance Overhead by Protection Tier

Table VIII places these measurements in end-to-end context across the four tiers. The hybrid design adds ~75–105 ms end-to-end while collapsing Vault RTO by ~99.8%.

TABLE V
IoT Prototype Test Results

Test case	Platform	TPM	Result
Boot-time unseal	RPi 4	SLB9670	OK; ~280 ms (e)
TOTP/HOTP under load	RPi 5	swtpm	OK; ~68 ms avg (e)
PCR enforcement	ARM64	fTPM	Tampered boot blocked (m)
Counter rollback	all	phys.+virt.	0 / 10,000 rollbacks (m)
Offline sealed cred	RPi 4	SLB9670	24 h TTL enforced (e)

TABLE VI
Provisioning Budget for the PPGIA96 Production Stack

Phase	Cold (first run)	Warm (cached)
Image pull + local builds	~3–6 min (e)	0 s (cached)
Container start to healthy	~15–30 s (e)	~15–30 s (e)
Vault sys/init (5/3 shares)	~1–2 s (e)	— (init.)
TPM seal shares + token	~12.2 ms/obj (m)	—
TPM auto-unseal (RTO)	~300 ms (e)	~300 ms (e)
ZTNA connector register	~2–5 s (e)	~2–5 s (e)
Total to production	**~3.5–6.5 min (e)**	**~25–45 s (e)**

TABLE VII
Secret-Retrieval and OTP Latency (software path, 20,000 iters)

Operation	Mean	p95	Source
TOTP generation (client)	0.009 ms	0.009 ms	(m) pyotp .now()
TOTP verification (server)	0.018 ms	0.019 ms	(m) pyotp .verify
Seed read — warm (cache)	0.0002 ms	0.0003 ms	(m) in-proc cache
Seed read — cold (Vault KV)	~8–12 ms	~15 ms	(e) hvac KV v2 LAN
TPM health check	6.98 ms	7.98 ms	(m) tpm2_getrandom
TPM seed provisioning (once)	12.22 ms	—	(m) createprimary+seal

VII. DISCUSSION

A. Limitations of the LLM Red-Team

The LLM adversary gives reproducible, MITRE-mapped regression testing, but (i) depends on prompt quality and a two-tool set; (ii) the turn limit constrains multi-step chains; (iii) 0% at n=100 carries a 95% Wilson upper bound of 3.6%, not absolute zero; (iv) it augments rather than replaces human red-teams.

B. Two-Channel MFA for IoT

Devices combine a TPM-bound identity (Channel 1, non-cloneable) with an out-of-band approval (Channel 2), both within a 30 s window aligned to the RFC 6238 time-step. An attacker must defeat the TPM (physically infeasible without silicon tampering [6]) and compromise the operator's out-of-band channel.

C. An Additional Point: Supply-Chain Integrity and Reproducibility

Beyond runtime defenses, buildtime trustworthiness matters: the whole stack is declared as code and every image is pinned (`vault:1.13.3`, `twingate/connector:1`, `eclipse-mosquitto:2`), so the deployment is reproducible and the test suite runs against the exact provisioned artifact rather than a hand-configured snapshot. We recommend signing images and Terraform plans (Sigstore/cosign) and storing attestations next to TPM PCR quotes, so both the build and the boot of each host are independently verifiable—future work.

D. Practical Considerations

Standard TPM 2.0 chips offer ~10 KB NVRAM, requiring careful index allocation. Intel deprecated SGX on 11th/12th-gen consumer CPUs (remains on server Xeon). Twingate needs its cloud controller and is unsuitable for fully air-gapped sites; self-hosted alternatives exist at different trade-offs.

VIII. CONCLUSION

We presented a hybrid Zero Trust architecture that mitigates OS-level credential extraction in nIA for IaC and IoT by anchoring Vault initialization in a physical TPM, sealing per-device OTP seeds in hardware, and confining every credential within an identity-based network perimeter. A publicly released reference implementation realizes the design; new in this revision, we characterized its provisioning time and secret-retrieval latency and proposed a repeatable MITRE-mapped security-test suite that gates each deployment. The software path costs microseconds; the hardware path adds a bounded, single-digit-to-tens-of-milliseconds overhead while cutting Vault RTO from minutes to milliseconds. Production-scale empirical validation remains the primary future work. All code is at github.com/juarez1972/app-tpm.

APPENDIX: FULL-WIDTH TABLES

TABLE III
Proposed Post-Deployment Security-Test Suite (after each terraform apply)

ID	Test (module / tool)	Technique	Pass criterion
A1	Port/service discovery (pentest.py, Nmap)	T1046 Service Discovery	No inbound port reachable except via ZTNA; Vault :8200 invisible from PPGIA95
A2	Authenticated vuln scan (pentest.py, OpenVAS)	CVE hygiene	No high/critical findings on exposed resources
A3	LLM red-team, cloud (pentestv2.py, Gemini)	Chained MITRE TTPs	Bypass rate within Wilson upper bound
A4	LLM red-team, on-prem (pentestv3.py, Llama/Ollama)	Same, air-gapped	Within Wilson bound; no data egress
B1	Unseal under PCR mismatch	T1542 Pre-OS Boot tamper	TPM refuses unseal; no seed in RAM
B2	Monotonic-counter rollback	T1550 stale-counter replay	Counter never decreases; stale code rejected
B3	TOTP replay within/after window	T1040/T1550 sniff-and-replay	Reused code rejected outside ±1 window
B4	Vault read without token/policy	T1552 Unsecured Creds	403; unreadable without app-policy
B5	Seed-at-rest disk/image scan	T1003 Credential Dumping	No plaintext seed on client disk or server code
B6	ZTNA token expiry / relogin (7d lock)	T1078 Valid Accounts	Expired session denied; reauth forced
B7	Reuse of leaked TOTP from outside ZTNA	T1021 Remote Services	Resource unreachable; code unusable off-net

TABLE IV
MITRE ATT&CK Adversary Simulation (n=100/technique; 500 runs total)

Tactic	Technique	SW-only	Hybrid (95% Wilson CI)	Primary mitigation
TA0001	T1078 Valid Accounts	8%	0% [0, 3.6%]	ZTNA session + TPM device binding
TA0004	T1068 Priv. Escalation	15%	0% [0, 3.6%]	SGX/TDX isolation of critical tier
TA0006	T1003 Cred. Dumping	22%	0% [0, 3.6%]	Non-exportable TPM keys; no seed at rest
TA0006	T1552 Unsecured Creds	35%	0% [0, 3.6%]	Vault policy scoping + ZTNA
TA0008	T1021 Remote Services	12%	0% [0, 3.6%]	Micro-segmentation + out-of-band MFA
	Average (SW-only)	**18.4%**	—	—

TABLE VIII
Performance Overhead by Protection Tier (projected; SW-path measured)

Operation	SW-only	T1 TPM	T2 SGX	T3 TDX	Added
HOTP/TOTP generation	~2 ms	~65 ms	~85 ms†	~70 ms†	+63–83 ms
Vault auth flow	~45 ms	~110 ms	~130 ms†	~115 ms†	+65–85 ms
End-to-end nIA	~120 ms	~195 ms	~225 ms†	~205 ms†	+75–105 ms
Auto-unseal RTO	~3 min	~300 ms	~350 ms†	~320 ms†	−99.8%

† SGX/TDX analytically estimated from published overhead [18] and Gramine benchmarks; software-path components measured directly (Table VII).

REFERENCES

[1] G. Deng et al., "PentestGPT: An LLM-empowered automatic penetration testing tool," arXiv:2308.06782, 2024.

[2] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with LLMs," Proc. ACM ESEC/FSE, 2023.

[3] A. Rahman, C. Parnin, L. Williams, "Security smells in Ansible and Chef scripts," ACM TOSEM, 30(1), 2021.

[4] S. K. Basak et al., "Practices for secret management in software artifacts," IEEE SecDev, 2022.

[5] A. Patel et al., "Dynamic secret injection for microservices in the cloud," ICICT, 2025.

[6] J. A. Halderman et al., "Lest we remember: Cold boot attacks on encryption keys," USENIX Security, 2008.

[7] D. M'Raihi et al., "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, 2005.

[8] J. Xu et al., "AutoAttacker: An LLM-guided system for automatic cyber-attacks," arXiv:2403.01038, 2024.

[9] MITRE Corporation, "ATT&CK matrix for enterprise," 2024. attack.mitre.org.

[10] HashiCorp, "Vault documentation: Auto-unseal," 2024.

[11] A. Shamir, "How to share a secret," Commun. ACM, 22(11):612-613, 1979.

[12] J. Oliveira, A. O. Santin, E. K. Viegas, P. Horchulhack, "A non-interactive one-time password-based method to enhance the Vault security," AINA 2024, LNDECT 202, Springer, 2024, pp. 201-213.

[13] Trusted Computing Group, "TPM 2.0 Library Specification, parts 1-4," Rev. 1.59, 2019.

[14] S. Rose et al., "Zero Trust Architecture," NIST SP 800-207, 2020.

[15] S. Teerakanok et al., "Migrating to zero trust architecture: reviews and challenges," Secur. Commun. Netw., 2021.

[16] V. Costan and S. Devadas, "Intel SGX explained," IACR ePrint 2016/086, 2016.

[17] P.-C. Cheng et al., "Intel TDX demystified: a top-down approach," arXiv:2303.15540, 2023.

[18] L. Coppolino et al., "An experimental evaluation of TEE technology evolution (SGX, SEV, TDX)," Computers & Security, 2025.

[19] C. Anuganti, "Federated DevOps: a zero-trust framework for privacy-preserving CI/CD," IJSRSET, 12(2), 2025.

[20] D. M'Raihi et al., "TOTP: Time-based one-time password algorithm," IETF RFC 6238, 2011.

[21] M. Bellare, R. Canetti, H. Krawczyk, "Keying hash functions for message authentication," IETF RFC 2104, 1997.

[22] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," JASA, 22(158):209-212, 1927.

[23] Twingate, "Deploy a connector via Docker," 2024.